★ PROJECT X-RAY ★

Cross-IDE, Multi-Language Codebase Intelligence & Visualization Platform

DOCUMENT TYPE	Detailed Project Report / Technical Synopsis
PROJECT	Project X-Ray
PREPARED BY	Saurav Kumar Bichha
QUALIFICATION	B.Tech (Computer Science & Engineering) Graduate — 2026
INSTITUTION	Noida International University, India
RESIDENCY	Nepali Resident
VERSION	1.1 — Professional Figures Edition
DATE	28 August 2026

A complete project report, technical synopsis and product concept document prepared as an original software engineering project proposal.

This document presents Project X-Ray as a proposed product and research-oriented software engineering project. It is not presented as an already completed commercial product, patent grant, or official university submission.

AUTHOR PROFILE & PROJECT OWNERSHIP

Prepared personally by the project proposer

I am Saurav Kumar Bichha, a B.Tech (Computer Science & Engineering) graduate of 2026 from Noida International University, India, and a Nepali resident. I am preparing Project X-Ray as an independent software engineering and product-development concept based on my interest in full-stack development, software architecture, developer tools and practical automation.

My initial technical strength is Java full-stack development. Therefore, the first implementation is intentionally designed around Java and Spring Boot rather than attempting to support every programming language immediately. The long-term architecture, however, is designed to expand to multiple programming languages and development environments.

The purpose of this report is to document the idea before implementation: what problem it solves, how it can work, what technologies are required, how it can become cross-platform, what risks exist, how it can be evaluated, and how it can eventually become a commercial developer-product platform.

• Project proposer: Saurav Kumar Bichha

• Academic background: B.Tech (CSE), 2026

• University: Noida International University, India

• Initial technical focus: Java, Spring Boot, full-stack development

• Project category: Developer tooling / software engineering / codebase intelligence

• Initial target: Java + Spring Boot projects

• Long-term target: multi-language, cross-IDE codebase intelligence

DECLARATION

Original project concept and responsible-use statement

I, Saurav Kumar Bichha, declare that this document describes my proposed Project X-Ray concept and development plan. The project is intended as an original software engineering effort developed by me, while recognizing that individual techniques such as source-code parsing, dependency graphs, static analysis, Git history analysis, IDE extensions and visualization are established areas of software engineering.

I do not claim in this report that every individual feature is globally novel or that no similar product exists. A formal prior-art, competitor and patent search would be required before making such claims. The intended novelty of Project X-Ray is to be established through its specific technical architecture, unified code model, cross-IDE design, explainable project-level analysis and the particular workflows implemented during development.

The project is intended to process source code responsibly. The preferred initial design is local-first analysis so that repositories do not need to be uploaded to an external server. Any future cloud or AI functionality should be explicitly optional and clearly disclosed to users.

> **NOTE**
> This declaration is a project statement, not legal advice and not a patent application.

ABSTRACT

Project X-Ray in simple terms

Project X-Ray is a proposed codebase intelligence and visualization platform that helps developers understand software projects at the project level rather than only at the individual-file level. A large codebase often contains hundreds or thousands of files, multiple modules, APIs, services, databases, tests and external dependencies. Understanding the relationships among these components can require substantial manual effort.

Project X-Ray addresses this problem by scanning a repository, extracting structural information, building a unified internal representation, and presenting the result through interactive visualizations and searchable reports. A developer could select a service and see which controllers call it, which repositories it uses, which entities are involved, which tests relate to it, and which other components may be affected by a change.

The first practical implementation is proposed for Java and Spring Boot. A Java analysis engine can identify packages, classes, interfaces, methods, annotations, imports and relationships. A Spring-specific layer can recognize common architectural roles such as controllers, services, repositories and entities. The result can be displayed through a VS Code extension, while the core analysis engine remains independent so that an IntelliJ Platform plugin, CLI and future standalone interface can reuse the same intelligence.

The long-term Project X-Ray platform can add dependency impact analysis, API visualization, code-health analysis, Git-based project history, project memory, learning mode, bug investigation and a broader Developer OS concept. The project is therefore designed as a scalable product architecture rather than a single IDE extension.

NOTE
Keywords: codebase intelligence, static analysis, software architecture, dependency analysis, developer tools, IDE extension, Java, Spring Boot, Git, visualization.

TABLE OF CONTENTS

Document navigation index

Table 1 — Table of Contents

LIST OF FIGURES, TABLES & KEY TERMS

Quick reference

The following index identifies the seven principal technical figures and every named table used throughout this report.

• Unified Code Model (UCM): common representation of source-code entities and relationships.

• Analyzer: a component that extracts structured information from a repository.

• Adapter: a language- or IDE-specific component that translates native information into the common model.

• Impact analysis: analysis of components potentially affected by changing a selected component.

• Local-first: source code is analyzed locally by default rather than uploaded to a cloud service.

1. INTRODUCTION & BACKGROUND

Why Project X-Ray is needed

Software development has moved from small collections of files toward large systems composed of services, libraries, APIs, databases, cloud resources and multiple programming languages. A developer can understand a single class quickly, but understanding the entire system can be much harder.

The challenge becomes especially visible when a developer joins an existing project. The developer must identify the application entry points, business modules, service relationships, database access patterns, external integrations and test structure. Documentation may be incomplete or outdated, while the source code contains the most current information.

Project X-Ray is based on a simple idea: the source code already contains many relationships that can be extracted automatically. Instead of asking a developer to manually reconstruct those relationships, the tool should build a useful project map and let the developer explore it interactively.

The project therefore sits between static analysis, software architecture visualization, developer productivity and IDE tooling. It is not intended to replace an IDE or a developer's judgment. It is intended to make the existing structure easier to discover.

2. PROBLEM DEFINITION

Current developer pain points

The central problem is not that developers cannot read source code. The problem is that a large software system distributes its meaning across many files and layers. A developer may understand each file individually while still lacking a reliable mental model of how the complete system behaves.

Common questions can require repeated manual navigation: Which controller reaches this service? Which services depend on this class? Which database tables are connected to this feature? What will probably be affected by a change? Which parts of the project have become unusually complex? What changed over the last six months?

Project X-Ray converts these questions into navigable project-level views. The goal is not to produce a decorative diagram. The diagram should be connected to real source files, symbols, Git information and measurable analysis results.

• Fragmented understanding across many source files

• Difficulty onboarding into unfamiliar repositories

• Manual tracing of dependencies and request flows

• Limited project-wide visibility of architecture

• Difficulty estimating change impact

• Architecture documentation becoming outdated

• Large Git histories being difficult to interpret

• Different IDEs providing different experiences

• Need for local processing when source code is sensitive

3. PROJECT VISION & SCOPE

From a Java MVP to a developer platform

The vision of Project X-Ray is to become a reusable intelligence layer for software projects. The product should answer structural questions about a codebase and present the answers in a form that developers can explore.

The initial scope is deliberately narrow: Java and Spring Boot, local analysis, Git integration, basic architecture visualization and VS Code integration. This creates a realistic starting point for implementation.

The long-term scope is much broader. The same engine can support multiple language analyzers, an IntelliJ Platform plugin, command-line workflows, CI reports, a web dashboard and optional team functionality.

Table 2 — Project Scope: MVP vs Later / Optional Features

In scope for MVP	Later / optional
Java source analysis	Python, Go, Rust, C#, C/C++
Spring Boot recognition	Advanced framework recognition
Architecture map	3D/advanced visualizations
Basic dependency graph	Advanced impact simulation
Git history	Full Code Time Machine
VS Code integration	IntelliJ + CLI + Web
Local storage	Optional cloud/team workspace

NOTE
The project should expand only after the first implementation demonstrates useful accuracy, performance and user value.

4. OBJECTIVES & REQUIREMENTS

Functional, non-functional and user requirements

The project should be judged by whether it saves developers time while maintaining trustworthy analysis. The system must therefore combine useful automation with transparency. When a result is directly measured from source code, it should be presented as an analysis result. When a result is an inference, the interface should make that clear.

The architecture should also be maintainable. New language support should be added through analyzers/adapters rather than by rewriting the visualization layer.

Table 3 — MVP Functional Requirements

ID	Requirement	Priority
FR-01	Scan a selected repository	Must
FR-02	Detect Java project structure	Must
FR-03	Build class/method relationships	Must
FR-04	Recognize Spring Boot roles	Must
FR-05	Show architecture graph	Must
FR-06	Navigate graph node to source	Must
FR-07	Show dependency relationships	Must
FR-08	Use Git history	Should
FR-09	Generate project report	Should
FR-10	Support additional languages	Future

Table 4 — Quality and Non-Functional Targets

Quality	Target
Accuracy	High precision on supported constructs
Performance	Responsive on small/medium repositories
Privacy	Local-first by default
Reliability	Graceful handling of incomplete code
Maintainability	Modular analyzers and adapters
Compatibility	Version-aware IDE integrations

5. SYSTEM CONCEPT & USER EXPERIENCE

How a developer would use Project X-Ray

The user experience should be simple. The developer opens a project and chooses 'Analyze Project'. X-Ray scans the workspace, identifies the technology stack, builds its internal model and opens the main project view.

The main screen should not overwhelm the user with every symbol in the repository. It should start at a high level and allow progressive exploration. A developer can move from system → module → service → class → method → source file.

Each visual element should remain connected to source code. Clicking a node should open the corresponding file and, where possible, focus the relevant symbol.

6. HIGH-LEVEL SYSTEM ARCHITECTURE

Core engine separated from IDE integrations

A central design decision is to separate the Project X-Ray engine from the IDE. The engine contains the project intelligence, while each IDE integration provides platform-specific commands, file navigation and user interface integration.

This approach avoids rebuilding the analysis logic separately for VS Code and IntelliJ. It also makes a command-line application possible because the same engine can analyze a repository without an IDE.

Figure 1 presents the proposed separation between the IDE/client layer, the Project X-Ray core engine, the data/knowledge layer and external integrations.

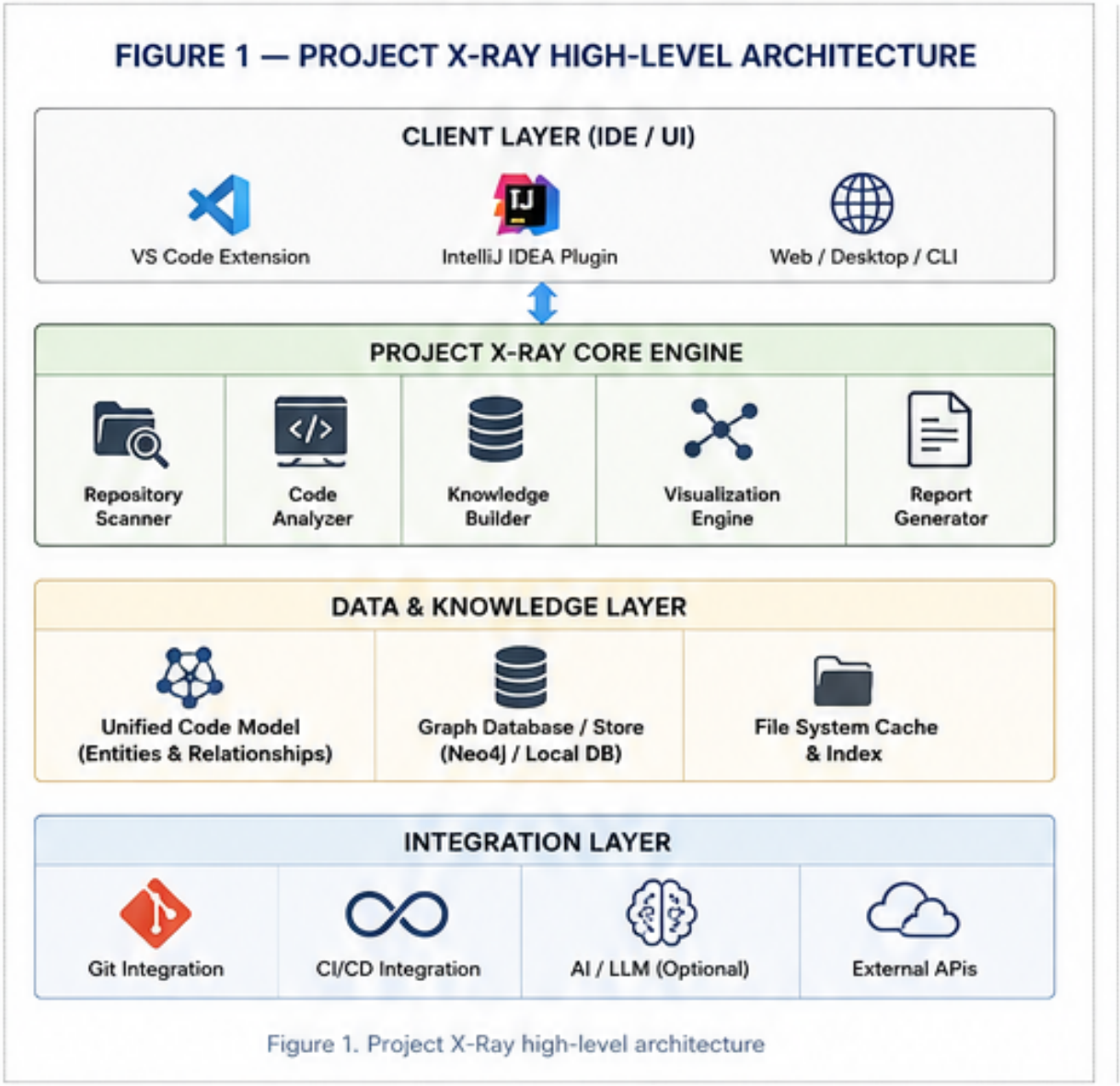

Figure 1 — Project X-Ray High-Level Architecture

Table 5 — High-Level Architecture Layers and Responsibilities

Layer	Responsibility
Workspace layer	Files, folders, build files, Git repository
Language layer	Parsing and language-specific extraction
Framework layer	Spring Boot and later framework recognition
Model layer	Unified Code Model

Layer	Responsibility
Analysis layer	Dependencies, APIs, health, impact
Presentation layer	IDE views, graphs, reports
Integration layer	VS Code, IntelliJ, CLI and future web

7. CODE ANALYSIS ENGINE

How source code becomes structured information

The analysis engine is the technical heart of Project X-Ray. It receives a repository and produces structured entities and relationships. For the first version, Java source can be parsed into an abstract syntax representation. The analyzer can then extract packages, classes, interfaces, methods, fields, annotations, imports, inheritance and method-level references where reliably available.

The engine should also inspect build configuration such as Maven or Gradle files. This allows it to understand external dependencies and project modules. Git metadata can be processed separately and connected to the same model.

The system should tolerate incomplete code. Developers often analyze projects while they are being edited, so a parser failure in one file should not necessarily stop the entire scan. The engine should record partial results and clearly identify files that could not be analyzed completely.

Figure 2 illustrates the repository-to-knowledge pipeline, from repository discovery and parsing through model construction, storage and visualization.

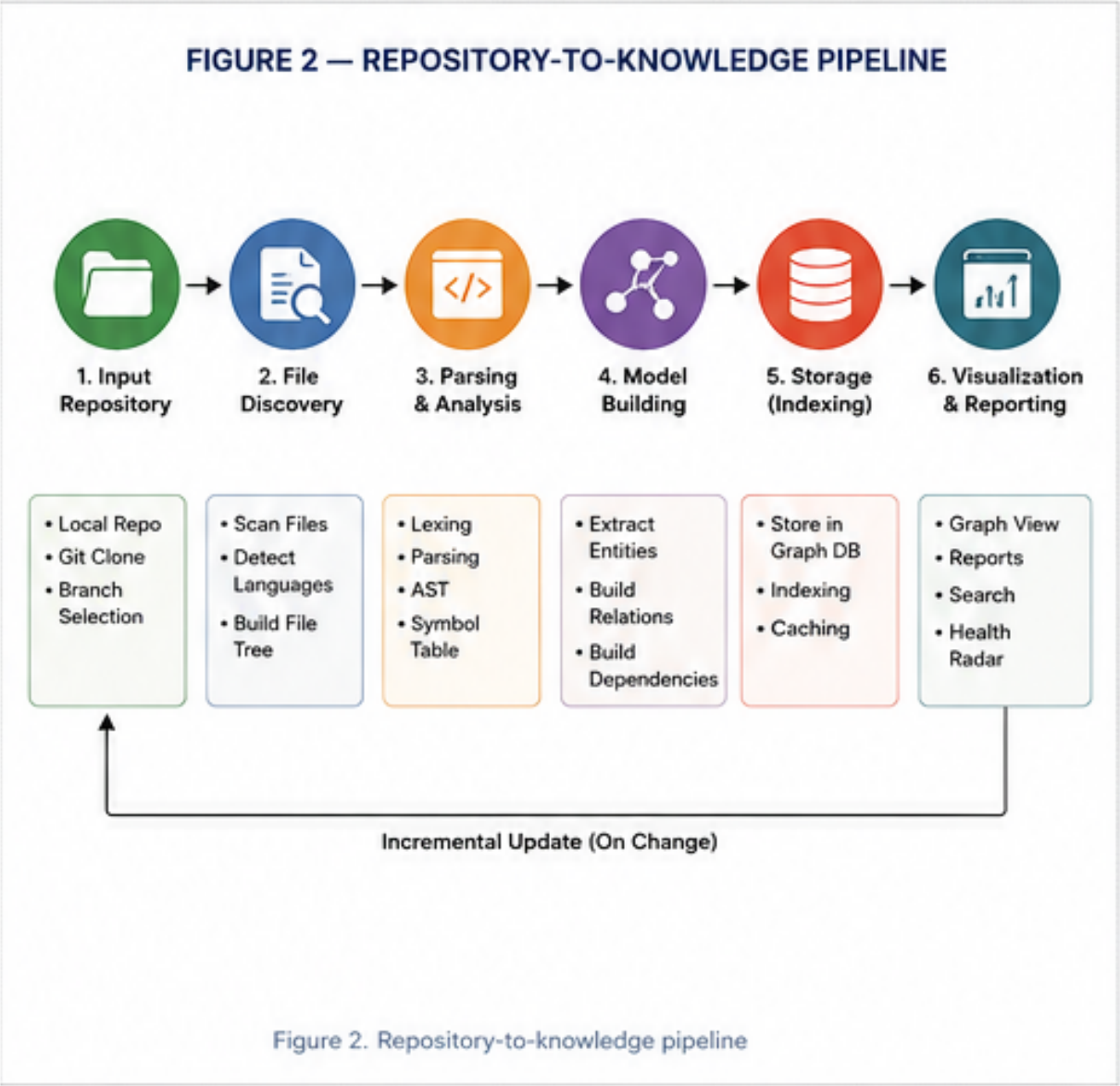

Figure 2 — Repository-to-Knowledge Pipeline

8. UNIFIED CODE MODEL

Common representation for multiple programming languages

To support multiple languages, Project X-Ray should not make its UI depend directly on Java-specific syntax. Instead, the analyzers produce a common model. A class in Java, a class in Python and a class in another object-oriented language may have different syntax, but they can still be represented as a code entity with a name, location, members and relationships.

The unified model should preserve language-specific details rather than hiding them. A generic entity can have a type such as class, interface, function, module, endpoint or database object, while an additional metadata section records language-specific information.

Table 6 — Unified Code Model Entities and Example Fields

Entity	Example fields
Project	name, rootPath, buildSystem
Module	name, path, language
Type	class/interface/struct/module
Function	name, parameters, return type
Endpoint	method, route, controller
Dependency	source, target, kind
Test	framework, target, file
DatabaseObject	table, schema, relation
GitEvent	commit, author, timestamp, files

NOTE
The Unified Code Model is a proposed internal design, not an established industry standard.

9. JAVA + SPRING BOOT MVP

First implementation strategy

The first implementation should target the technology stack that I can develop most effectively: Java full-stack and Spring Boot. This reduces the initial learning burden and makes it possible to focus on the core product problem.

The Spring-specific analyzer can identify common annotations such as controller mappings, service components, repository components and entity definitions. It can then connect these components into a likely request-to-data flow.

The MVP should not attempt to understand every Spring feature. It should support a defined set of common constructs, document the supported cases, and gracefully report unsupported or ambiguous constructs.

Figure 3 shows the representative Spring Boot request path that the first analyzer is intended to reconstruct.

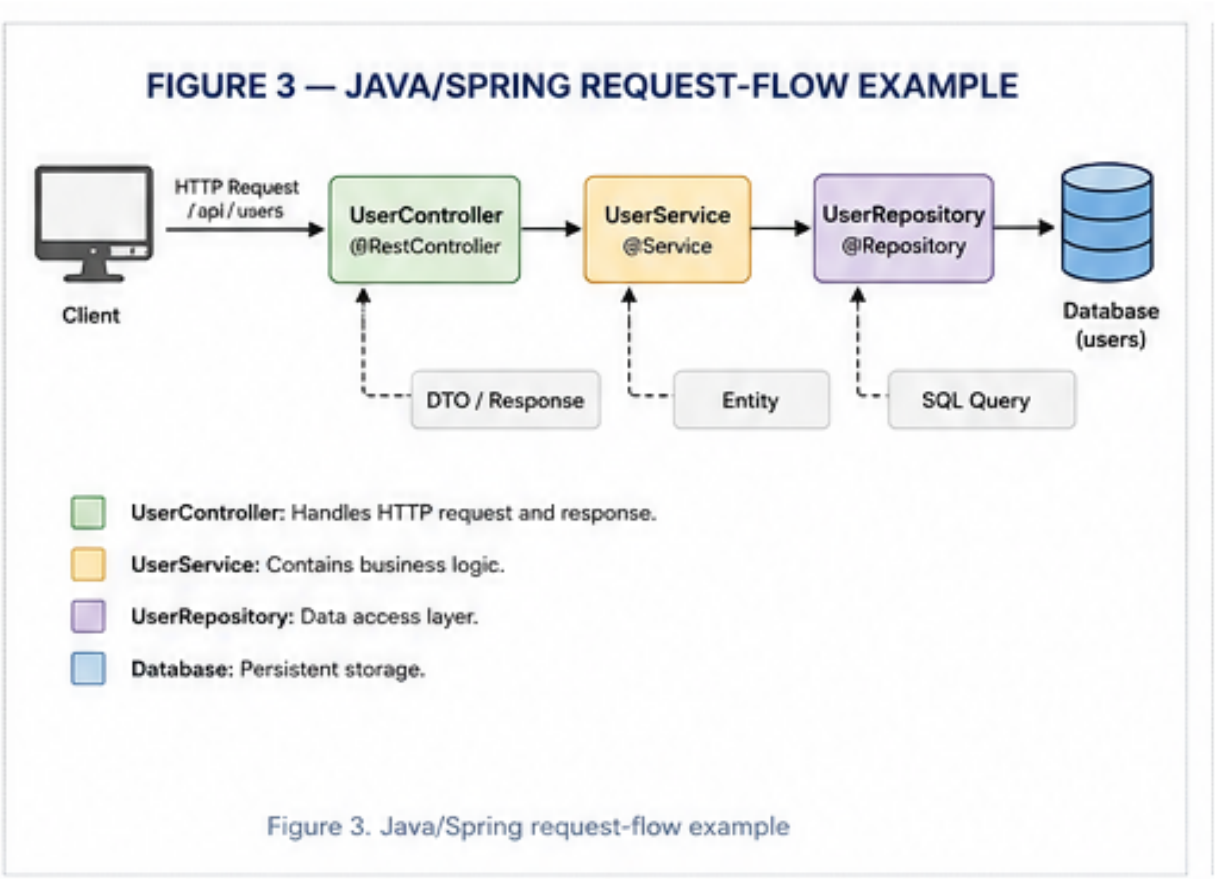

Figure 3 — Java/Spring Request-Flow Example

Table 7 — Initial Java/Spring Boot Technology Stack

Technology	Initial role
Java	Primary source language
Spring Boot	Framework-aware analysis
Maven/Gradle	Project/build dependency discovery
JUnit	Test relationship discovery
Git	Change history
TypeScript/Node.js	VS Code integration
SQLite/files	Local analysis cache

10. VISUALIZATION & USER INTERFACE

Turning analysis into an understandable experience

The visual interface is central to Project X-Ray because the product is intended to reduce cognitive load. A raw list of thousands of classes is not enough. The UI should provide multiple levels of detail: overview, module, component, dependency and source-level views.

A graph should be interactive. Users should be able to zoom, filter, search and select nodes. The interface should avoid showing every relationship at once because dense graphs become unreadable.

A useful design is progressive disclosure: show important architectural components first, then reveal details when a node is selected.

- Architecture map
- Module explorer
- Dependency graph
- API flow view
- Selected-component detail panel
- Search and filter
- Open source action
- Git history panel
- Code-health panel
- Exportable project report

> **NOTE**
> For VS Code, custom UI can be implemented using supported extension mechanisms such as webviews and custom editors where appropriate. Webviews should be used only when native APIs are insufficient because they carry additional performance and UX considerations.

11. DEPENDENCY GALAXY & CHANGE-IMPACT ANALYSIS

Understanding what may be affected by a change

Dependency analysis is one of the most commercially valuable parts of Project X-Ray. A developer often wants to know what might be affected before modifying a shared component.

The first version can calculate direct relationships such as imports, inheritance and known references. Later versions can include framework-level relationships and call graphs where analysis quality permits.

The result should be presented as potential impact, not certainty. Static analysis can identify relationships, but runtime behavior, reflection, configuration and external systems can make actual impact more complex.

Figure 4 illustrates how a selected component can be traced through dependencies, tests, APIs and data objects to produce a potential-impact report.

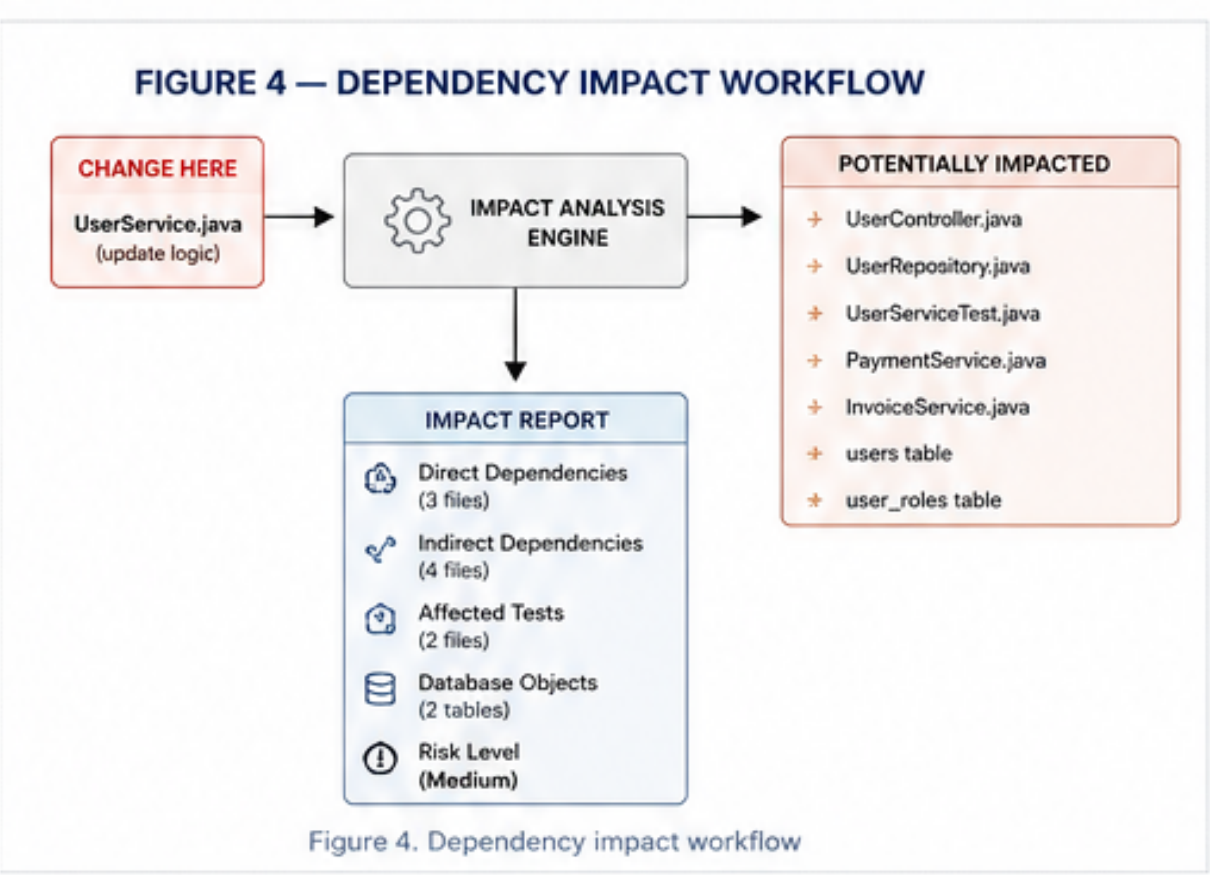

Figure 4 — Dependency Impact Workflow

Table 8 — Dependency Impact Signals

Impact signal	Meaning
Direct dependency	Known source-level relationship
Transitive dependency	Relationship through another component
API connection	Endpoint/controller path reaches component
Test relation	Test appears connected to component
Recent change	Component changed recently
High coupling	Large number of connected components

12. API UNIVERSE

From endpoint to business logic and data

For backend applications, an API view can be more useful than a class graph. Project X-Ray should eventually build an API Universe that starts from an HTTP endpoint and traces the internal flow.

For Spring Boot, this can begin with request-mapping annotations and controller methods. The system can then follow known service and repository relationships and show the likely data layer connection.

The API view should support reverse exploration too: selecting a service could show which API endpoints can reach it.

• Endpoint discovery

• HTTP method and route

• Controller mapping

• Service path

• Repository/data path

• Authentication annotations where detectable

• Related tests

• OpenAPI/specification integration as a future module

13. CODE HEALTH RADAR

Measurable software quality signals

Project X-Ray can provide a health view based on measurable signals rather than a single mysterious score. Developers should be able to inspect why an area is considered complex or risky.

Possible indicators include cyclomatic complexity, class size, method size, dependency count, coupling, duplication, test relationships and change frequency. The first version should avoid pretending that these numbers alone prove code quality.

A radar or dashboard can group signals by module and allow the developer to drill down to the exact files responsible for a metric.

Table 9 — Code Health Metrics and Interpretation

Metric	What it can indicate
Complexity	Logic that may be harder to reason about
Coupling	How strongly a component is connected to others
Duplication	Repeated code patterns
Change frequency	Areas that change often
Test relationship	Whether relevant tests appear to exist
Dependency count	Number of direct connections
Size	Large classes or methods requiring attention

NOTE
Health indicators should be treated as engineering signals, not absolute judgments.

14. CODE TIME MACHINE

Using Git to understand project evolution

Git contains a historical record of how the project changed. Project X-Ray can transform that record into a higher-level timeline rather than forcing developers to inspect individual commits manually.

The Code Time Machine concept would compare selected commits or periods and show changes in modules, dependencies, APIs and major components. The system could answer questions such as: when did the payment module appear, which components became more connected, and which parts have changed repeatedly?

The first implementation can use ordinary Git metadata and source snapshots. Advanced versions can create historical models at selected checkpoints and compare them.

15. BUG REPLAY & INVESTIGATION

Evidence-based debugging support

Bug Replay is a future module that combines code changes, tests, Git history and available diagnostics to help reconstruct the sequence around a failure.

The tool should never present a correlation as proven causation without evidence. Instead it can show a ranked investigation trail: files changed before the failure, related tests, dependency changes, and components connected to the failing area.

This feature is intentionally placed after the MVP because reliable runtime-level reconstruction is substantially harder than static code mapping.

• Collect relevant Git changes

• Connect failing tests to source components

• Identify recent changes near the failing area

• Show dependency context

• Create a chronological investigation view

• Allow the developer to mark the actual root cause after investigation

• Use the marked result for future project knowledge only when explicitly enabled

16. WORKSPACE MEMORY

Persistent, structured project knowledge

Workspace Memory is designed to store useful project knowledge separately from temporary AI chat history. The memory should belong to the project and be understandable to both humans and future tools.

Examples include architecture decisions, coding conventions, important modules, known constraints and reasons for design choices. The system can store structured records rather than a single large text document.

A local-first approach is preferable for the initial version. Teams could later choose a shared workspace with access control.

17. LEARNING MODE

Using a real codebase as an interactive learning environment

Learning Mode is a future feature intended for developers who are new to an existing project. Instead of reading files randomly, the user receives a guided route through the architecture.

For example, a Spring Boot project could be introduced as controller → service → repository → entity. The user can then open each component and receive a structured explanation of its role, dependencies and related files.

This feature can also be useful for junior developers, interns and teams onboarding new members, but it should not replace official project documentation.

- Project overview lesson
- Architecture walkthrough
- Module-by-module learning path
- Selected-class explanation
- Glossary of project-specific terms
- Practice questions based on the project
- Optional AI explanations with explicit user control

18. CROSS-IDE STRATEGY

VS Code, IntelliJ Platform and command line

Project X-Ray should not be permanently tied to one editor. The core engine should be independent. VS Code and IntelliJ integrations can act as clients of the same analysis layer.

VS Code provides an Extension API, custom UI capabilities and language-server patterns. JetBrains provides the IntelliJ Platform plugin architecture and language-specific modules. Compatibility must be tested against targeted platform versions because IDE APIs can change.

A CLI is important because it allows the engine to run in CI/CD and outside an IDE. This also provides a useful test surface for the core engine.

Figure 6 shows the intended separation between IDE clients, the reusable Project X-Ray core engine and project data sources.

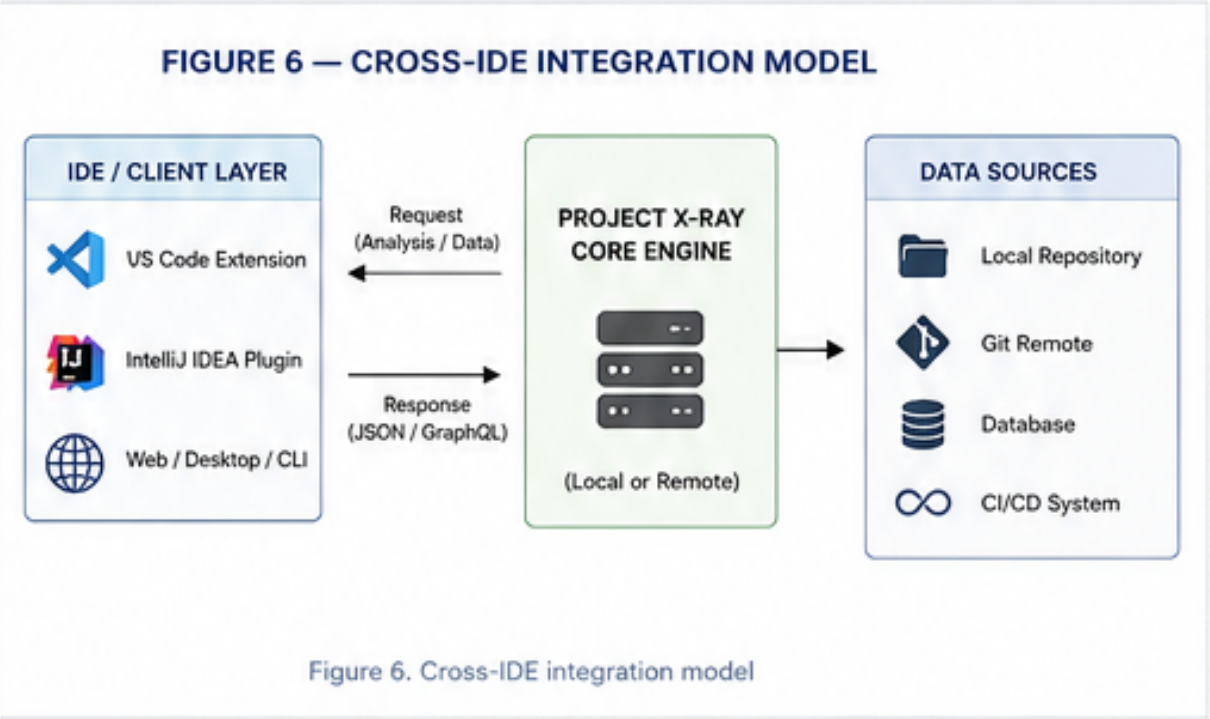

Figure 6 — Cross-IDE Integration Model

Table 10 — Cross-IDE Integration Strategy

Platform	Role
VS Code	First IDE integration and MVP UI
IntelliJ Platform	Second IDE integration, especially valuable for Java developers
CLI	Automation, CI and standalone analysis
Web	Future dashboards and team views
Desktop	Optional future standalone application

> **NOTE**
> Official documentation confirms that VS Code supports extensions and custom UI, while JetBrains documents platform modules and plugin compatibility across IntelliJ-based products. These integrations should be version-tested rather than assuming every IDE version is identical.

19. MULTI-LANGUAGE STRATEGY

How Project X-Ray can eventually support many languages

Project X-Ray can support many programming languages, but support should be incremental. The common model allows each language analyzer to translate language-specific structures into shared entities and relationships.

Java is the first target because it matches my existing full-stack background. TypeScript/JavaScript and Python are logical next steps because they are common in modern full-stack and backend projects. Kotlin, Go, Rust and C# can follow.

Some concepts are language-specific. Therefore the system should never force every language into an identical representation. The common model should contain generic fields plus language-specific metadata.

Figure 5 shows how language-specific adapters can feed different programming-language constructs into the shared Unified Code Model.

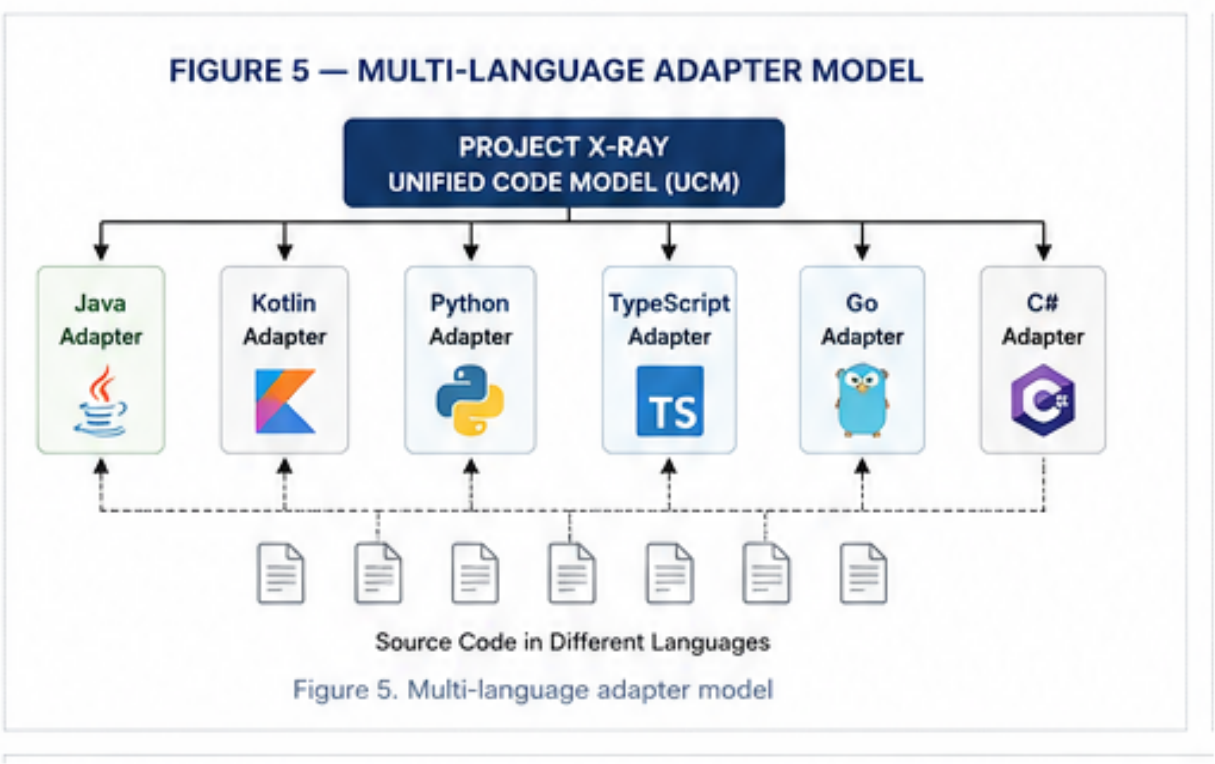

Figure 5 — Multi-Language Adapter Model

Table 11 — Language Support Roadmap

Stage	Languages / ecosystems
1	Java + Spring Boot
2	TypeScript + JavaScript
3	Python + Kotlin
4	Go + Rust + C#
5	C/C++ + PHP + Ruby + Swift + Dart
6	Mixed-language repositories

20. SECURITY, PRIVACY & RESPONSIBLE DESIGN

Protecting source code and developer activity

Source code is often confidential intellectual property. A responsible Project X-Ray design should therefore default to local analysis. The MVP should not require a user to upload source code to a remote service.

If future features use cloud AI or shared team workspaces, the product should clearly state what information leaves the machine. Sensitive files such as environment files, secrets and credential stores should be excluded by default from analysis.

The optional Developer Replay Recorder also requires special care. Recording developer activity should be opt-in, transparent and easy to disable. Data should be deletable and should not silently capture secrets.

- Local-first analysis
- Explicit cloud/AI opt-in
- Secret-file exclusion
- Project-level ignore configuration
- Encrypted shared data if cloud features are introduced
- Clear retention and deletion controls
- No hidden activity recording
- Audit-friendly logs for team features

21. TECHNOLOGY STACK & DEVELOPMENT ENVIRONMENT

Practical stack for the first implementation

The stack is intentionally based on free or open-source development tools for the MVP. The main investment is development time and learning rather than software licensing.

Table 12 — Initial Technology Stack and Development Environment

Component	Technology	Reason
Core engine	Java	Existing skill and strong ecosystem
Build	Maven or Gradle	Java project management
Source analysis	Java parsing / AST tooling	Structured code extraction
Framework analysis	Spring-aware rules	First target ecosystem
Version control	Git	History and change analysis
Local storage	SQLite / project files	Simple local persistence
VS Code	TypeScript + Node.js	Extension integration
UI	HTML/CSS/JS or React	Interactive visualization
IntelliJ	IntelliJ Platform plugin	Cross-IDE support
CLI	Java command-line layer	CI and standalone analysis
Testing	JUnit + integration tests	Engine validation

22. DETAILED MODULE DESIGN

Proposed internal project structure

The core module contains reusable intelligence. Language-specific modules implement parsers and framework rules. The IDE modules translate user actions into engine queries and provide source navigation.

This modular design also makes testing easier. The core engine can be tested independently from VS Code or IntelliJ. A sample repository can be used as a fixed analysis fixture, allowing expected relationships to be compared with actual results.

• Scanner: discovers repository structure

• Model: stores normalized entities

• Parser: converts source into syntax structures

• Analyzer: derives relationships

• Dependency engine: builds graph

• API engine: discovers endpoints and flows

• Health engine: calculates metrics

• Git engine: builds historical information

• Storage: caches results locally

• Query layer: serves IDE/CLI requests

23. DATA MODEL & EXAMPLE SCHEMA

How X-Ray could store project intelligence

A relational or document-style local store can represent the analysis cache. The exact database design should be chosen after the first prototype. For an MVP, a small relational schema is easy to inspect and debug.

Table 13 — Project X-Ray Logical Data Model

Table	Key fields
projects	id, name, root_path, language
modules	id, project_id, name, path
symbols	id, module_id, type, name, file, line
relations	source_id, target_id, relation_type
endpoints	id, symbol_id, method, route
tests	id, symbol_id, framework, file
git_events	id, commit, timestamp, message
metrics	symbol_id, metric_name, value

NOTE
This is a proposed logical schema. The implementation may use SQLite tables, JSON indexes, an embedded graph representation, or another suitable local storage design.

24. DEVELOPMENT ROADMAP

Step-by-step implementation plan

The first public MVP should be released only after the core analysis is reliable enough to avoid misleading developers. It is better to support fewer constructs accurately than to claim broad support with unreliable results.

Table 14 — Development Roadmap and Expected Outcomes

Phase	Main work	Expected outcome
0	Research + architecture	Technical specification
1	Java scanner + parser	Basic source model
2	Spring analyzer	Controller/service/repository mapping
3	Graph + source navigation	First X-Ray visualization
4	Dependency impact	Change exploration
5	Git integration	History view
6	Health radar	Quality dashboard
7	VS Code polish	Usable MVP
8	IntelliJ plugin	Second IDE
9	CLI + CI	IDE-independent usage
10	More languages	Multi-language platform

25. TESTING & EVALUATION METHODOLOGY

How the project should be validated

Testing should include both software correctness and usefulness to developers. A graph that looks attractive but contains incorrect relationships is not a successful code-intelligence tool.

A controlled test suite can contain small repositories with known architecture. Each repository should have expected entities and relationships. The analyzer output can be compared against those expected results.

User evaluation can measure whether developers understand an unfamiliar project faster with X-Ray than with normal file navigation alone. This can be a later academic research study if appropriate.

Table 15 — Proposed Testing and Evaluation Metrics

Metric	Example measurement
Entity accuracy	Correctly identified classes/modules
Relation precision	Percentage of reported relationships that are valid
Relation recall	Percentage of known relationships discovered
Scan time	Seconds/minutes per repository
Memory use	Peak RAM during analysis
Navigation benefit	Time to locate relevant components
Onboarding benefit	Time to explain architecture
Stability	Crashes/failures per scan

26. BUSINESS MODEL & MONETIZATION

How Project X-Ray could become a product

The first version can be free so that developers can try the tool and provide feedback. A commercial model can later be introduced around advanced analysis, larger repositories, team functionality and optional cloud services.

The product should not charge simply for opening a graph. Paid features should solve meaningful professional problems such as advanced impact analysis, historical architecture, team dashboards, CI reports and organization-wide code intelligence.

A possible structure is Free, Pro and Team/Business. Pricing should be tested with actual users rather than fixed prematurely.

Table 16 — Possible Product Tiers and Features

Tier	Possible features
Free	Local scan, basic architecture, basic dependencies
Pro	Advanced impact, history, health analytics, more languages
Team	Shared workspace, CI reports, architecture monitoring
Business	Organization controls, SSO/access policies, support, private deployment

> **NOTE**
> Pricing examples are product hypotheses, not market-validated prices.

27. COMPETITIVE POSITIONING & DIFFERENTIATION

Where Project X-Ray should compete

Project X-Ray should not position itself simply as another AI coding assistant. The stronger positioning is codebase understanding: helping developers see the structure, relationships, evolution and measurable health of a project.

Existing IDEs and developer tools already provide many individual capabilities. The proposed differentiation is the combination of a reusable core model, visual project-level exploration, cross-IDE architecture and progressive expansion into dependency, API, history and project-memory features.

A formal competitor study should be conducted before commercialization. This report intentionally avoids claiming that the overall idea is globally unique.

- Project-level rather than only file-level understanding
- One core engine with multiple IDE integrations
- Language adapters instead of one-language lock-in
- Evidence-linked visualizations
- Local-first privacy
- Explainable metrics
- Architecture evolution through Git
- Potential CI and team workflows

28. IP / PATENT & PRIOR-ART STRATEGY

How to protect the project responsibly

The word 'patent' should be used carefully. A project idea is not automatically patentable merely because it is useful or implemented in software. Patentability depends on applicable law, novelty, inventive step/non-obviousness and other requirements.

Before a patent filing, I should preserve dated design documents, technical experiments and implementation records and conduct a professional prior-art search. The search should examine developer tools, static analysis, architecture visualization, code intelligence, dependency analysis and related research.

If a genuinely distinctive technical mechanism emerges, a patent professional can assess whether that mechanism is suitable for protection. Copyright, trademarks, trade secrets and controlled disclosure may also be relevant depending on what is developed.

• Maintain dated technical specifications

• Record implementation milestones

• Do not publicly disclose potentially sensitive invention details before professional advice if patent protection is being considered

• Perform prior-art and competitor research

• Identify the actual technical mechanism claimed as novel

• Seek qualified IP advice before filing

• Do not describe a project report as a granted patent

NOTE
This section is informational and not legal advice.

29. RISKS, LIMITATIONS & MITIGATION

Realistic assessment of the project

The largest technical risk is false confidence. A tool that looks authoritative but produces incorrect architecture or impact results can be worse than having no tool. Accuracy, transparency and the ability to inspect evidence should therefore be core product principles.

Table 17 — Risk Register and Mitigation Plan

Risk	Impact	Mitigation
Incorrect relationships	High	Precision-focused analyzers + confidence labels
Large repositories	High	Incremental indexing + caching
Language complexity	High	Start with one language
IDE API changes	Medium	Version testing and adapters
UI graph overload	Medium	Filtering + progressive disclosure
Privacy concerns	High	Local-first architecture
AI hallucination	High	Evidence-linked explanations
Low user adoption	High	Early developer testing
Scope creep	High	Strict MVP milestones
Maintenance burden	High	Modular architecture

30. FUTURE ROADMAP — FROM X-RAY TO DEVELOPER OS

Long-term product evolution

The long-term vision is not to release ten unrelated extensions. The ten previously discussed concepts can become modules of one developer platform. Project X-Ray is the foundation because it creates the structured representation of the codebase.

Dependency Galaxy, API Universe and Code Health Radar can consume the same model. Code Time Machine adds historical models. Workspace Memory adds human/project knowledge. Learning Mode and bug investigation use the same relationships to create higher-level workflows.

This modular strategy gives the project a realistic path from a student-built MVP to a professional developer tool without requiring the entire platform to exist on the first day.

Figure 7 presents the proposed product evolution from the Java/Spring Boot MVP toward a multi-language Developer OS vision.

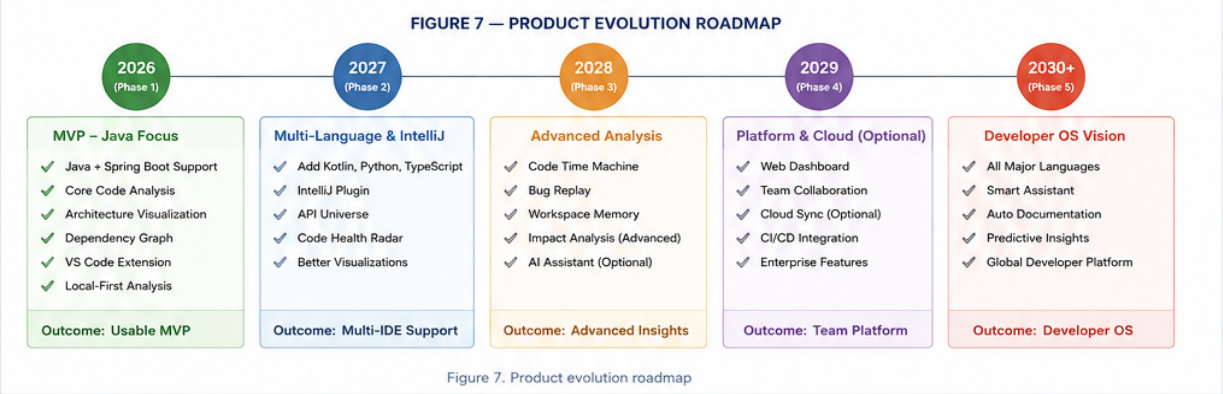

Figure 7 — Product Evolution Roadmap

31. EXPECTED CONTRIBUTION

Technical, academic and practical value

From a technical perspective, Project X-Ray combines source-code analysis, software architecture modeling, graph-based visualization, Git analysis and IDE integration into a coherent developer workflow.

From an academic perspective, the project can support research questions around codebase comprehension, visualization, change-impact analysis and developer onboarding. Controlled experiments can compare conventional navigation with X-Ray-assisted exploration.

From a practical perspective, the project can become a usable tool for developers working with unfamiliar repositories. Its strongest initial niche is Java/Spring Boot, where the first analyzer can be developed deeply before the platform expands.

- A reusable code-intelligence core
- A practical Java/Spring Boot analyzer
- Interactive architecture visualization
- Potential cross-IDE integration
- Measurable software-quality indicators
- A foundation for future commercial developer tooling

32. CONCLUSION

Final project statement

Project X-Ray is proposed as a codebase intelligence and visualization platform that helps developers understand software systems as connected structures rather than isolated files. The project begins with a realistic Java/Spring Boot MVP and is intentionally designed to grow into a multi-language, cross-IDE platform.

As Saurav Kumar Bichha, I can begin this project using my existing Java full-stack knowledge and gradually learn TypeScript, VS Code extension development and IntelliJ Platform integration. The development tools required for the MVP can largely be free or open source, making the project financially accessible at the prototype stage.

The most important technical principle is separation of concerns: the core analysis engine should remain independent from any particular IDE. This makes the same intelligence reusable across VS Code, IntelliJ-based IDEs, CLI workflows and future standalone interfaces.

The most important product principle is trust: X-Ray should show developers where its information came from, distinguish static facts from inference, handle uncertainty honestly and protect source code by default.

The immediate next step is to convert this report into a detailed implementation specification for Project X-Ray v0.1: repository structure, Java parser/analyzer modules, Unified Code Model, Spring Boot rules, local database/cache, VS Code extension commands, visualization UI, test repositories and milestone-by-milestone development tasks.

> **NOTE**
> End of main report.

APPENDIX A — INITIAL VS CODE EXTENSION DESIGN

Commands, views and interaction model

The VS Code extension should remain lightweight and delegate heavy analysis to the core engine. It can use supported VS Code extension mechanisms for commands, views and custom visual interfaces. A custom visual panel can communicate with the extension host using message passing.

The extension should avoid repeatedly scanning the entire repository after every small edit. Incremental updates and debouncing should be used where practical.

Table 18 — VS Code Extension Commands

Command	Action
Project X-Ray: Analyze Project	Start or refresh analysis
Project X-Ray: Open Overview	Open main architecture view
Project X-Ray: Find Component	Search symbols/modules
Project X-Ray: Analyze Impact	Analyze selected component
Project X-Ray: Open History	Open Git history view
Project X-Ray: Generate Report	Create project report

APPENDIX B — INTELLIJ PLATFORM DESIGN

Second IDE integration

The IntelliJ integration should be implemented as a separate plugin module. It can use IntelliJ Platform facilities where appropriate, while still keeping Project X-Ray's higher-level model independent.

JetBrains documentation describes shared IntelliJ Platform modules and product-specific language plugins. Plugin compatibility must therefore be declared and tested against target IDE/platform versions.

For Java projects, IntelliJ's existing program-structure facilities can provide useful information, while Project X-Ray's own model can normalize that information for cross-platform features.

Table 19 — IntelliJ Platform Plugin Components

Plugin component	Responsibility
Action	Run X-Ray commands
Tool window	Show overview and details
Editor integration	Navigate to symbols
Project listener	Detect relevant project changes
Engine bridge	Communicate with X-Ray core
Compatibility layer	Target supported platform versions

APPENDIX C — EXAMPLE PROJECT ANALYSIS

A small Spring Boot example

Suppose a project contains UserController, UserService, UserRepository and UserEntity. Project X-Ray identifies these components and builds logical relationships from the source code.

The visual result is intentionally simple at the first level. Selecting any component can reveal its file, symbol, dependencies, related tests and other known connections.

APPENDIX D — SAMPLE PROJECT-XRAY JSON MODEL

Illustrative internal representation

```
{
"project": "ExampleShop",
"language": "Java",
"framework": "Spring Boot",
"modules": ["api", "service", "data"],
"symbols": [
{"type": "controller", "name": "UserController"},
{"type": "service", "name": "UserService"},
{"type": "repository", "name": "UserRepository"}
],
"relations": [
{"from": "UserController", "to": "UserService", "type": "calls"},
{"from": "UserService", "to": "UserRepository", "type": "uses"}
]
}
```

NOTE
The schema is illustrative. The production model should be versioned and designed around actual analyzer requirements.

APPENDIX E — IMPLEMENTATION CHECKLIST

First implementation milestones

- Create Git repository and project README
- Create Java core module
- Create domain model
- Implement repository scanner
- Detect Maven/Gradle
- Parse Java source
- Extract packages, classes and interfaces
- Extract methods and imports
- Extract inheritance and annotations
- Create Spring annotation rules
- Detect controllers, services, repositories and entities
- Build relationship graph
- Persist local analysis cache
- Create CLI analyze command
- Create VS Code extension skeleton
- Add command to launch analysis
- Connect extension to engine
- Create overview panel
- Render architecture graph
- Add node selection and source navigation
- Add search and filters
- Add dependency details
- Add Git information
- Add initial health metrics
- Create sample test repositories
- Create expected-model fixtures
- Write unit and integration tests
- Measure scan time and memory
- Optimize caching
- Handle parser errors gracefully
- Write user documentation
- Run developer usability tests
- Fix misleading results
- Package the MVP
- Prepare release notes
- Plan IntelliJ adapter
- Plan the next language adapter

APPENDIX F — OFFICIAL TECHNICAL REFERENCES

Primary documentation used for the platform architecture

Microsoft Visual Studio Code Extension API — https://code.visualstudio.com/api/

Microsoft VS Code Webview API — https://code.visualstudio.com/api/extension-guides/webview

Microsoft VS Code Custom Editor API — https://code.visualstudio.com/api/extension-guides/custom-editors

Microsoft VS Code Language Server Extension Guide — https://code.visualstudio.com/api/language-extensions/language-server-extension-guide

JetBrains IntelliJ Platform Plugin SDK — https://plugins.jetbrains.com/docs/intellij/welcome.html

JetBrains Plugin Compatibility — https://plugins.jetbrains.com/docs/intellij/plugin-compatibility.html

JetBrains PSI documentation — https://plugins.jetbrains.com/docs/intellij/psi.html

These sources support the architectural statements about VS Code extensibility, custom UI, language-server patterns, IntelliJ Platform plugins, platform modules, PSI and compatibility testing.

NOTE
The supplied reference PDF was used as the formatting and organization inspiration for this report: formal cover, author/declaration material, navigation index, sectioned technical chapters, tables, diagrams, implementation roadmap and appendices. Its Nepal policy content is not reproduced here.